

## 第十二章作业

### 题目

#### 1. 在数据库中为什么要采用并发控制？并发控制技术能保证事务的哪些特性？

数据库是共享资源，多个事务经常需要同时访问数据库。若不对并发执行的事务进行控制，可能会出现数据丢失、数据不一致等问题，从而破坏数据库的正确性。

因此，数据库管理系统必须采用并发控制技术来协调各事务之间的执行顺序。

并发控制主要保证事务的：

- 一致性（Consistency）
- 隔离性（Isolation）

从而保证数据库始终处于正确状态。

---

#### 2. 并发操作可能会产生哪几类数据不一致？用什么方法能避免各种不一致的情况？

并发操作可能产生以下三类数据不一致问题：

##### （1）丢失修改（Lost Update）

两个事务同时修改同一数据，后提交事务覆盖先提交事务的结果。

##### （2）不可重复读（Non-Repeatable Read）

同一事务中两次读取同一数据，结果不同。

##### （3）读脏数据（Dirty Read）

一个事务读取了另一个尚未提交事务修改的数据。

避免这些问题的主要方法：

- 封锁（锁）机制
- 两段锁协议（2PL）
- 时间戳并发控制

- 多版本并发控制（MVCC）

### 3. 事务的隔离级别都有哪些，事务隔离级别与数据一致性的关系是什么？

数据库常用的四种事务隔离级别如下：

| 隔离级别             | 脏读 | 不可重复读 | 幻读 |
|------------------|----|-------|----|
| Read Uncommitted | 可能 | 可能    | 可能 |
| Read Committed   | 避免 | 可能    | 可能 |
| Repeatable Read  | 避免 | 避免    | 可能 |
| Serializable     | 避免 | 避免    | 避免 |

关系：

事务隔离级别越高，数据一致性越好，但系统并发性能越低；隔离级别越低，并发能力越强，但产生数据不一致的可能性越大。

### 4. 什么是封锁？基本的封锁类型有几种？试述它们的含义。

封锁是并发控制中最常用的技术，即事务在访问数据对象前先申请相应的锁。

基本封锁类型有两种：

#### （1）排他锁（X锁）

又称写锁。

获得X锁后：

- 本事务可读可写
- 其他事务不能读也不能写

#### （2）共享锁（S锁）

又称读锁。

获得S锁后：

- 本事务可读
- 其他事务也可读
- 其他事务不能写

---

## 5. 如何用封锁机制保证数据的一致性？

事务在读数据前申请S锁，在写数据前申请X锁。

通过锁的兼容与互斥规则：

- S锁与S锁兼容
- S锁与X锁冲突
- X锁与X锁冲突

从而保证多个事务不会同时修改同一数据，避免产生各种数据不一致问题。

---

## 6. 什么是活锁？试述活锁的产生原因和解决方法。

活锁是指事务一直处于等待状态，但系统始终没有进展的现象。

产生原因：

某事务长期被其他事务反复插队。

解决方法：

- 采用先来先服务（FIFO）策略
  - 设置等待队列
  - 限制事务优先级变化
- 

## 7. 什么是死锁？请给出预防死锁的若干方法。

死锁是指两个或多个事务互相等待对方释放资源，从而永远无法继续执行。

预防方法：

1. 一次申请全部资源。
  2. 资源按固定顺序申请。
  3. 采用时间戳法。
  4. 采用等待—死亡策略。
  5. 采用伤害—等待策略。
-

8. 请给出检测死锁发生的一种方法。当发生死锁后如何解除死锁？

检测方法：

建立等待图（Wait-For Graph）。

若等待图中出现环路，则说明发生死锁。

解除方法：

- 撤销一个事务；
- 回滚部分事务；
- 释放其占有资源；
- 重新调度执行。

9. 什么样的并发调度是正确的调度？

正确调度应满足：

并发执行结果与某种串行执行结果等价。

即该调度必须是：

- 可串行化调度（Serializable Schedule）

这样才能保证数据库一致性。

10. 设 T1、T2、T3 如下（A初值为0）。

T1:  $A=A+2$

T2:  $A=A\times 2$

T3:  $A=A^2$

(1) 可能的正确结果

六种串行执行顺序及结果：

| 顺序       | 结果 |
|----------|----|
| T1→T2→T3 | 16 |
| T1→T3→T2 | 8  |

| 顺序       | 结果 |
|----------|----|
| T2→T1→T3 | 4  |
| T2→T3→T1 | 2  |
| T3→T1→T2 | 4  |
| T3→T2→T1 | 2  |

正确结果集合：

{2,4,8,16}

## (2) 一个可串行化调度

T1 → T2 → T3

结果：

A=16

## (3) 一个非串行化调度

R1(A)  
R2(A)  
W1(A)  
W2(A)  
T3执行

该调度可能产生错误结果。

## (4) 遵守两段锁协议且不产生死锁

T1完成 → T2完成 → T3完成

### (5) 遵守两段锁协议且产生死锁

T1锁A等待B  
T2锁B等待A

形成循环等待。

## 11. 判断调度是否冲突可串行化。

调度：

R3(B) R1(A) W3(B) R2(B) R2(A) W2(B) R1(B) W1(A)

构造优先图：

T3 → T2  
T2 → T1

图中无环。

因此该调度：

是冲突可串行化调度

等价串行顺序：

T3 → T2 → T1

## 12. 证明若并发事务遵守两段锁协议，则这些事务的并发调度是可串行化的。

两段锁协议规定：

- 扩展阶段：只加锁不解锁；
- 收缩阶段：只解锁不加锁。

事务获得最后一个锁的时刻称为锁点。

按照各事务锁点先后顺序排列，可得到一个等价串行顺序。

因此遵守两段锁协议的调度一定是可串行化调度。

---

### 13. 举例说明可串行化调度不一定遵守两段锁协议。

例如：

```
S(A)
R(A)
UNLOCK(A)
S(B)
R(B)
```

事务在释放锁后又申请新锁。

该调度可能仍与某个串行调度等价，因此是可串行化调度。

但违反两段锁协议。

---

### 14. 说明下列调度集合之间的包含关系。

设：

- ① 正确调度
- ② 可串行化调度
- ③ 遵循两段锁协议的调度
- ④ 串行调度

则：

```
④ ⊂ ③ ⊂ ② ⊂ ①
```

即：

串行调度  $\subset$  两段锁调度  $\subset$  可串行化调度  $\subset$  正确调度

---

## 15. 考虑事务T1与T2。

### (1) 增加加锁与解锁操作

T1:

```
LOCK-S(A)
R(A)

LOCK-X(B)
R(B)
B=A+B
W(B)

UNLOCK(A)
UNLOCK(B)
```

T2:

```
LOCK-S(B)
R(B)

LOCK-X(A)
R(A)
A=A+B
W(A)

UNLOCK(B)
UNLOCK(A)
```

符合两段锁协议。

### (2) 是否会产生死锁

会。

例如：



T1获得A锁

T2获得B锁

T1等待B

T2等待A

形成循环等待。

因此可能产生死锁。

---

## 16. 为什么要引进意向锁？意向锁的含义是什么？

在多粒度封锁中，系统需要同时管理数据库、表、页、记录等多个层次。

若逐层检查所有下级节点是否加锁，效率很低。

因此引入意向锁。

意向锁表示：

“该节点的下层对象准备加锁”。

其作用是提高多粒度封锁管理效率。

---

## 17. 试述常用的意向锁（IS锁、IX锁、SIX锁），给出这些锁的相容矩阵。

### （1）IS锁

意向共享锁。

表示准备在下层对象加S锁。

### （2）IX锁

意向排他锁。

表示准备在下层对象加X锁。

### （3）SIX锁

共享意向排他锁。

表示当前对象加S锁，同时准备在下层对象加X锁。

相容矩阵

|     | IS | IX | S | SIX | X |
|-----|----|----|---|-----|---|
| IS  | √  | √  | √ | √   | × |
| IX  | √  | √  | × | ×   | × |
| S   | √  | ×  | √ | ×   | × |
| SIX | √  | ×  | × | ×   | × |
| X   | ×  | ×  | × | ×   | × |

思维导图

